

Level order traversal

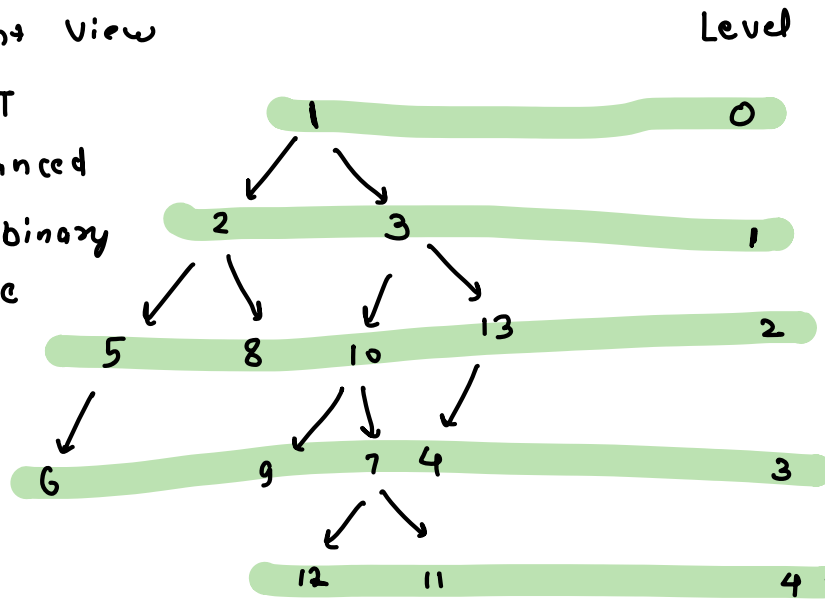
✓ → Level order traversal

✓ → Left & Right View

✓ → Types of BT

✓ → Height balanced

✓ → Construct binary tree



has to have deg nodes

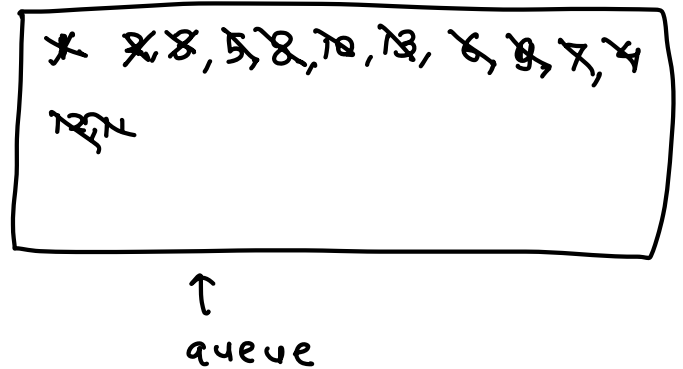
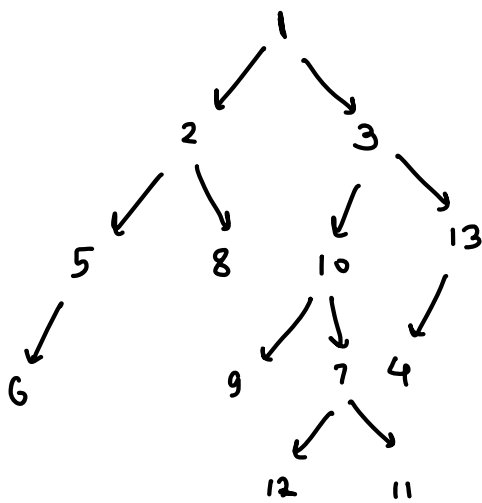
Level order traversal

1, 2, 3, 5, 8, 10, 13, 6, 9, 7, 4, 12, 11

print each level from top to bottom, [left to right by default for each level]

[Inorder
pre
post → Depth First
traversal]

[level order → Breadth First
traversal]



1, 2, 3, 5, 8, 10, 13, 6, 9, 7, 4, 12, 11

Queue < Node > q

Tc:

q.push(head)

while (q.isEmpty() == false)

Node tmp = q.front()

q.dequeue()

print (tmp.data)

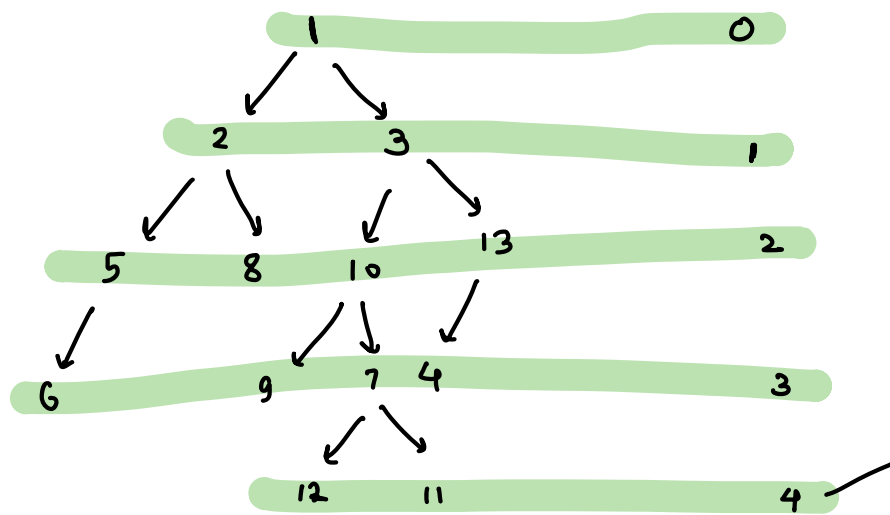
if (tmp.left != NULL) q.push (tmp.left)

if (tmp.right != NULL) q.push (tmp.right)

return

Tc: O(N)

Sc: O(N)



Level order traversal

1
 2, 3
 5, 8, 10, 13
 6, 9, 7, 4
 12, 11

Queue < Node > q

q.push(head)

while (q.isEmpty() == False)

int cnt = q.size()

for (j=1, j ≤ cnt; j++)

Node tmp = q.front()

q.dequeue()

print (tmp.data)

if (tmp.left != NULL) q.push (tmp.left)

if (tmp.right != NULL) q.push (tmp.right)

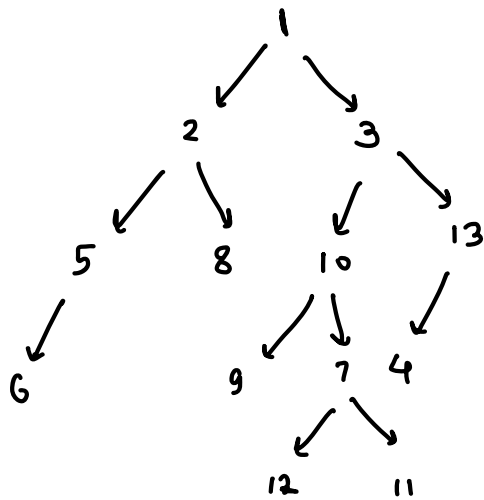
Tc: O(N)

Sc: O(N)

println ()

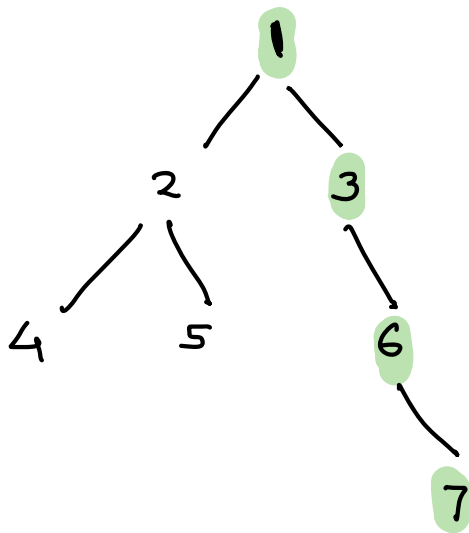
~~x~~, null, ~~x~~, ~~x~~, ~~null~~, ~~x~~, ~~x~~, ~~10~~, ~~13~~, ~~null~~, ~~x~~, ~~x~~, ~~x~~, ~~x~~, ~~null~~

~~10~~, ~~x~~, null



1
 → 2 3
 → 5 8, 10 13
 → 6 9 7 4
 → 12, 11

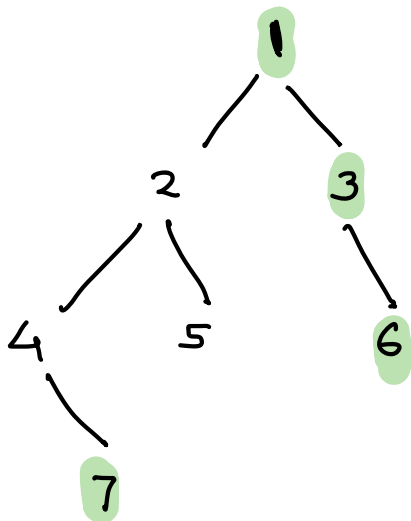
Q1.



Right view: What a person can see from the right side



Right view $\Rightarrow [1, 3, 6, 7]$



Right view $\Rightarrow [1, 3, 6, 7]$



$[1, 3, 6, 7, 10]$

Queue < Node > q

q.push(head)

Tc: $O(N)$
Sc: $O(N)$

while (q.isEmpty() == False)

int cnt = q.size()

for (j=1; j ≤ cnt; j++)

Node tmp = q.front()

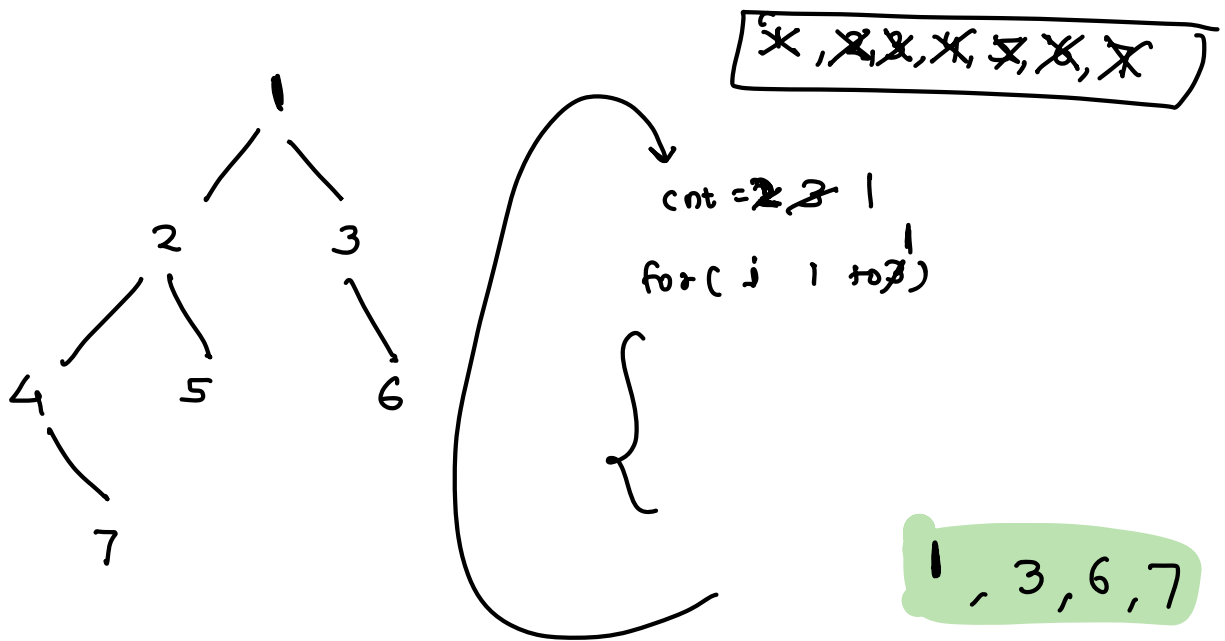
q.dequeue()

if (j == cnt) print (tmp.data)

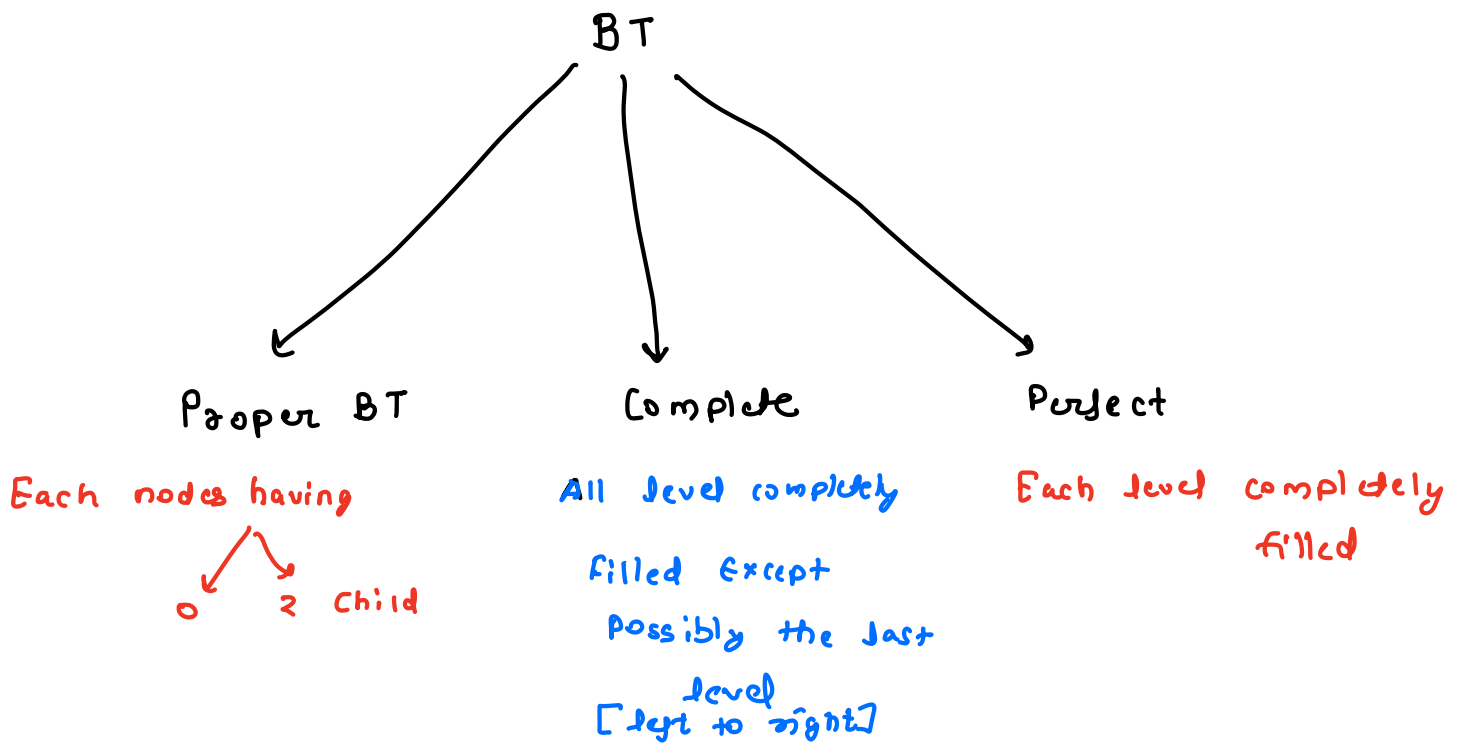
if (tmp.left != NULL) q.push (tmp.left)

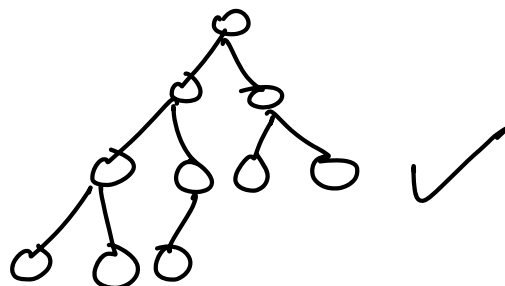
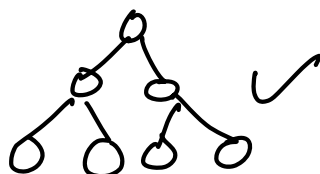
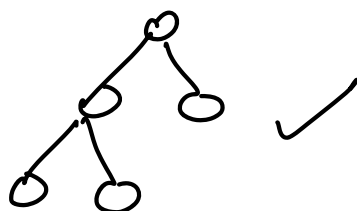
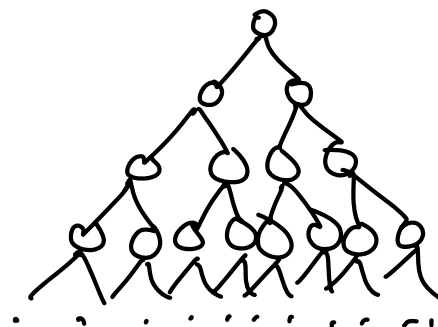
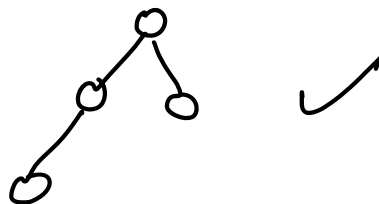
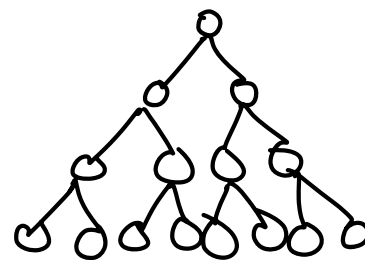
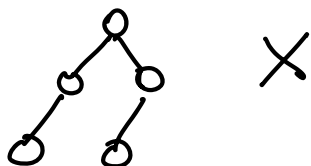
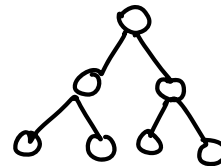
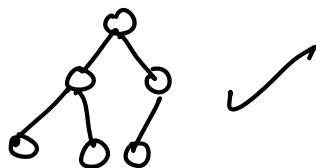
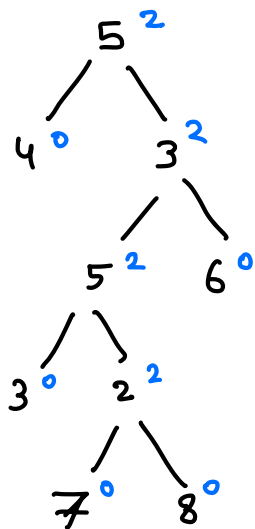
if (tmp.right != NULL) q.push (tmp.right)

println ()



Just view $\rightarrow$ First Node of each level





Perfect will be complete & proper

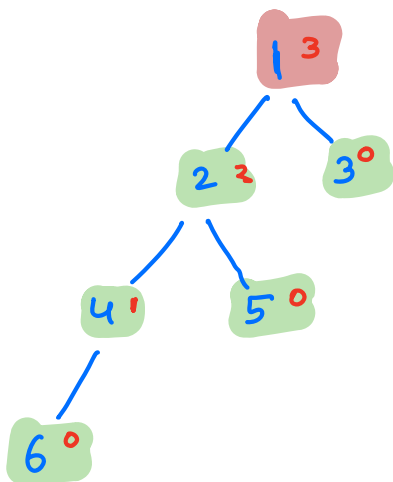
Complete will be perfect ~~X~~

Complete will be proper ~~X~~

Q. Given a BT. Check if it is height balanced or not.

For all nodes

$$abs(\text{height of left child} - \text{height of right child}) \leq 1$$



Not height balanced tree

Brute Force

bool isHeightBalanced (Node root)



Left height = height (root.left)

right " = " " ,right)

if (abs (Left height - right height) > 1)

return False

return isHeightBalanced (root.left)

&&

isHeightBalanced (root.right)

int height (root)



return 1 + max [height (root . left)
height (root . right)]

Tc: $O(N^2)$

Idea 2

int height (root , bool ans)

if (root == NULL) return -1

l = height (root . left)

r = height (root . right)

if (abs (l - r) > 1) ans = False

return 1 + max (l , r)

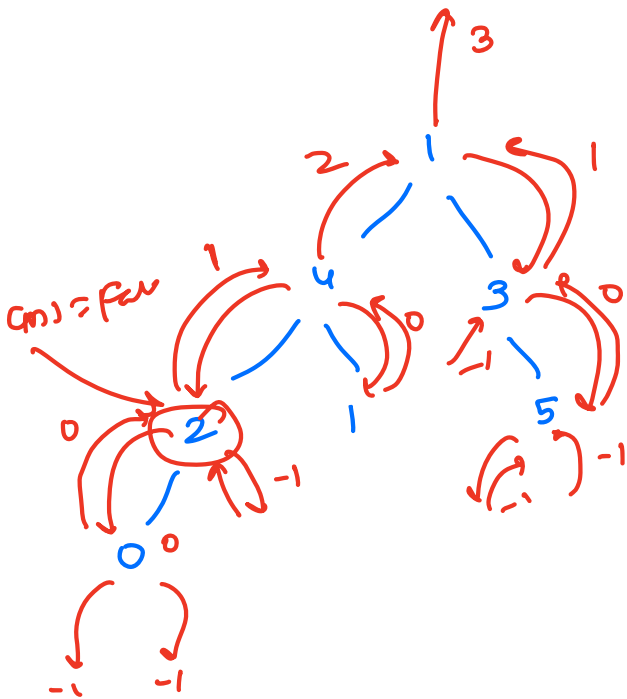
Sc: 0C NJ

bool can main()

ans = True

height (root, fans)

return ans


$$\text{height}(1) =$$

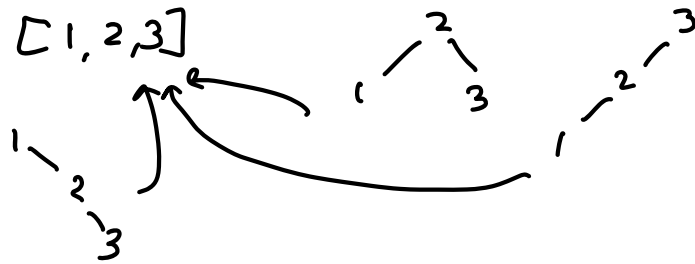
keig, At (3)

$$h_{L^2}, h_T(2)$$
$$h(c_1 h + c_4)$$

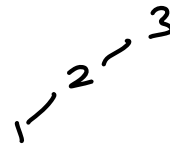
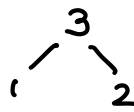
Flight (12)

Inorder

[1, 2, 3]



Postorder [1, 2, 3]

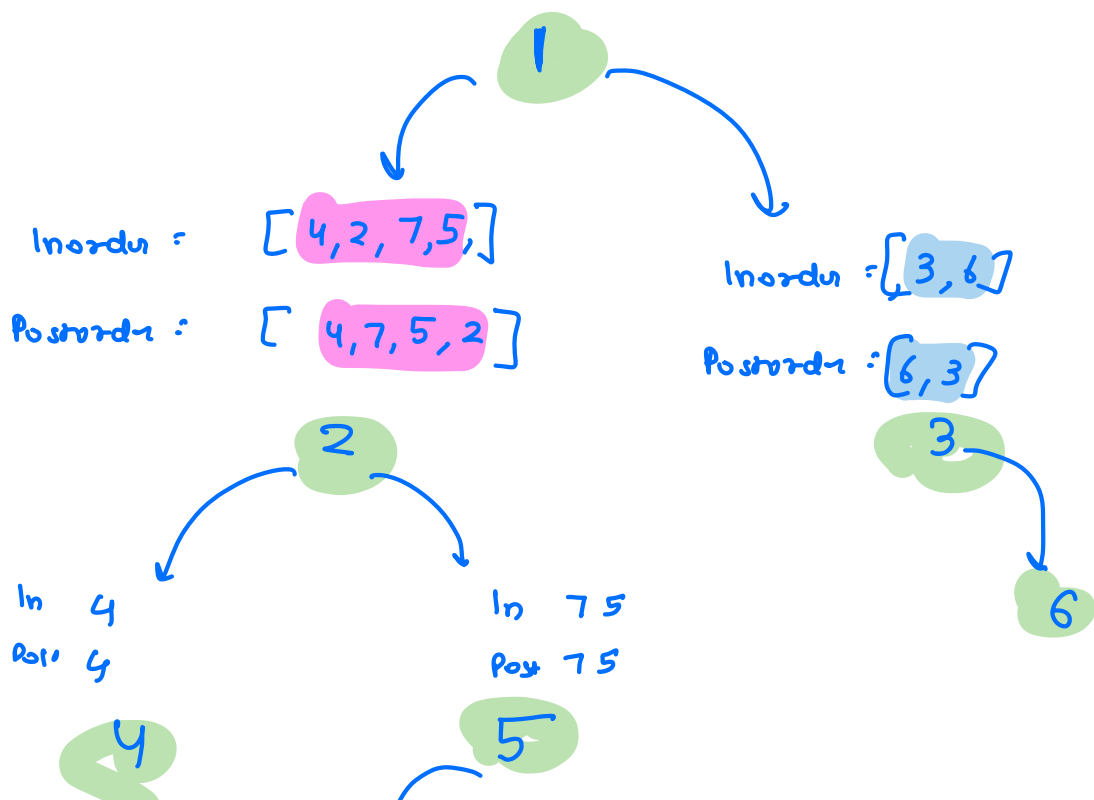


Q. Given Inorder & Postorder of a BT. [unique elements]

Construct BT

Inorder: [4, 2, 7, 5, 1, 3, 6]

Postorder: [4, 7, 5, 2, 6, 3, 1]



Node buildTree (In[], Post[])

// hashmap of In[]

mp[element] = index

return f (In, Post, 0, n-1, 0, n-1)

↑ construct & return the root

Node f (In[], Post[], is, ie, ps, pe)

if (is > ie) return null

root element = Post[pe]

Node tmp = new Node (root element)
rootind

for (j = is ; j ≤ ie ; j++)

{ if (In[j] == root element)
{ rootind = j
break }

tmp.left = f (In, Post, is, rootind-1, ps, x)

tmp.right = f (In, Post, rootind+1, ie, x+1, pe-1)

return tmp

hm.get (root element)

① Queue

